



Intent becomes action.

Engineering Work Breakdown

4-Week Sprint · Maaz · Nekha · Ashish · Minimum Merge Conflicts

1. File Ownership Map — Conflict Prevention

This is the most important section. Each file and directory has a single owner. The other engineer imports from it but never modifies it. Merge conflicts are nearly eliminated when these boundaries hold.

Rule:

If you need to change a file owned by the other engineer, open a PR discussion first. Do not commit to someone else's file unilaterally. The one exception: SHARED contracts defined in Week 1 — both engineers agree on those types before parallel work begins.

File / Directory	Owner	Notes / Conflict Guard
<code>lib/intent/types.ts</code>	SHARED	Defined by Maaz in W1 Day 1. Nekha imports — never modifies.
<code>lib/intent/parser.ts</code>	Maaz	Core NLP — intent extraction for all 7 service types
<code>lib/intent/participants.ts</code>	Maaz	@handle resolution, invite generation for non-iAM users
<code>lib/intent/graph.ts</code>	Maaz	Intent graph updates post-booking — outcome signals

<code>lib/intent/profileQuery.ts</code>	Maaz	Profile visitor prompt answering, scoped to owner context
<code>lib/ranking/gate.ts</code>	Maaz	Relevance threshold — most sensitive file in codebase
<code>lib/ranking/scorer.ts</code>	Maaz	Intent Fit, User Fit, Outcome History scoring
<code>lib/ranking/ranker.ts</code>	Maaz	Bid applied only within already-qualified set
<code>lib/ranking/types.ts</code>	Maaz	ScoreCard, RankingContext, BidInput types
<code>lib/checkout/types.ts</code>	SHARED	Defined by Maaz in W1 Day 2. Nekha imports CartItem, StageCart.
<code>lib/checkout/pendingOrder.ts</code>	Maaz	PendingOrder creation, expiry, status management
<code>lib/checkout/split.ts</code>	Maaz	Vendor split — idempotent Stripe Connect transfers
<code>lib/payments/stripe.ts</code>	Maaz	Stripe client, PaymentIntent, SplitPayment modes
<code>lib/services/flights/</code>	Maaz	Finish existing partial integration
<code>lib/profile/docParser.ts</code>	Maaz	Document upload parsing — maps to intent graph fields
<code>app/api/intent/route.ts</code>	Maaz	Intent submission handler
<code>app/api/checkout/route.ts</code>	Maaz	Checkout initiation, cart validation
<code>app/api/webhooks/stripe/route.ts</code>	Maaz	Stripe webhook — payment_intent.succeeded
<code>app/api/profile/upload/route.ts</code>	Maaz	Document upload endpoint, Vercel Blob
<code>app/api/profile/prompt/route.ts</code>	Maaz	Profile visitor prompt handler
<code>app/[handle]/page.tsx</code>	Maaz	Profile page — Nekha provides privacy middleware only
<code>lib/db/mongo.ts</code>	Nekha	Exports getDb(). Maaz calls getDb() only — never edits schema here.
<code>lib/services/duffel/client.ts</code>	Nekha	Shared Duffel HTTP client. Maaz's flights module imports this.
<code>lib/services/stays/</code>	Nekha	Duffel hotel search, room offers, booking
<code>lib/services/cars/</code>	Nekha	Duffel car rental search, category filter, booking
<code>lib/services/viator/</code>	Nekha	Viator product search, availability, booking
<code>lib/services/opentable/</code>	Nekha	OpenTable availability search, reservation creation

<code>lib/services/maps/</code>	Nekha	Google Maps Places API — POIs, malls, shopping
<code>lib/stage/assembler.ts</code>	Nekha	Promise.all() across all 7 services, imports ParsedIntent
<code>lib/stage/merge.ts</code>	Nekha	Multi-participant intent graph merge and card ranking
<code>lib/cache/</code>	Nekha	All Redis caching — TTL strategy per service
<code>lib/sse/</code>	Nekha	SSE infrastructure, broadcast, reconnection handling
<code>lib/gifts/</code>	Nekha	GiftOrder model, token generation, create and redeem logic
<code>app/api/gifts/</code>	Nekha	Gift create and redeem API routes
<code>app/gift/[token]/page.tsx</code>	Nekha	Gift reveal page — item display, address form
<code>app/api/stage/sse/route.ts</code>	Nekha	Persistent SSE connections per participant
<code>app/api/stage/lock/route.ts</code>	Nekha	Item locking API with MongoDB persistence
<code>app/api/stage/stream/route.ts</code>	Nekha	Progressive row streaming to frontend
<code>lib/privacy/</code>	Nekha	Profile privacy settings, data sharing rules, follow requests
<code>app/api/follow/</code>	Nekha	Follow request create, accept, decline, list
<code>store/stageStore.ts</code>	Nekha	Stage structure and row data
<code>store/participantStore.ts</code>	Nekha	Participant presence, status, follow state
<code>store/cartStore.ts</code>	Nekha	Locked cards per participant — real-time sync

2. Shared Type Contracts — Defined Week 1, Day 1–2

Both engineers must agree on these interfaces before parallel work begins. Once merged, they are frozen until explicitly renegotiated. Neither engineer modifies these unilaterally after Day 2.

`lib/intent/types.ts` — Maaz defines, Nekha imports

Type	Fields Nekha depends on
<code>ParsedIntent</code>	<code>destination</code> , <code>dates</code> , <code>participants[]</code> , <code>activityTypes[]</code> , <code>budgetSignal</code>

ActivityType	enum: flights stays cars experiences restaurants weather maps
Participant	handle, userId (nullable), intentGraph (nullable)
MergedStageContext	participants[], sharedIntent: ParsedIntent, mergedGraph: IntentGraph

lib/checkout/types.ts — Maaz defines, Nekha imports

Type	Fields Nekha depends on
CartItem	id, cardId, vendorId, vendorType, amount, currency, lockedBy, isShared, bookingPayload
StageCart	stageId, participants[], items: CartItem[], status, paymentMode
VendorType	enum: duffel_flight duffel_stay duffel_car viator opentable
PendingOrder	id, stageId, vendorItems[], totalAmount, payerId, status, expiresAt

lib/db/mongo.ts — Nekha defines, Maaz uses getDb() only

Export	What Maaz uses
getDb()	MongoDB connection — Maaz calls this in lib/intent/, lib/checkout/, lib/profile/
Collection names	COLLECTIONS.users, COLLECTIONS.orders, etc. — constants Maaz imports
ensureIndexes()	Called once on startup — Nekha owns all index definitions

Day 2 Gate:

Neither engineer begins parallel feature work until both shared type files are merged to main and both engineers have pulled. This one half-day investment eliminates the majority of possible merge conflicts.

3. 4-Week Gantt

Week	Maaz	Nekha	Ashish
------	------	-------	--------

W1 Foundation	Freeze ParsedIntent + CartItem types (Day 1-2). Finish lib/intent/parser.ts for all 7 activity types. Finish lib/services/flights/ — offer expiry, error handling. lib/ranking/ — gate.ts, scorer.ts, ranker.ts skeleton. lib/payments/stripe.ts — client + PaymentIntent base.	lib/db/mongo.ts — all collections, indexes, getDb(). lib/services/duffel/client.ts — shared HTTP client. lib/services/stays/ — hotel search + room offers complete. lib/services/cars/ — car search + category filter. lib/cache/ — Redis TTL strategy, caching wrapper.	Design system audit — tokens, dark mode coverage, component gaps. Stage row + card anatomy (all 7 service types). Profile page wireframes (header, bio, prompt). Gift reveal page wireframe. Onboarding questions UI.
W2 Integrations	lib/intent/participants.ts — @handle resolution + invite link generation. app/api/intent/route.ts. lib/profile/docParser.ts — document parser. app/api/profile/upload/route.ts. lib/ranking/ — scorer.ts full implementation with unit tests.	lib/services/viator/ — search, availability, booking. lib/services/opentable/ — availability, reservation. lib/services/maps/ — Places API, POIs, malls. lib/stage/assembler.ts — Promise.all() all 7 services. lib/stage/merge.ts — multi-participant graph merge.	Stage UI — 7 row types, card variants, progressive loading states. Locking animation + participant lock indicators. Real-time presence bar (who's in the Stage). Profile page full UI — header, avatar, prompt input area. Document upload UX.
W3 Checkout + Collaboration	lib/checkout/pendingOrder.ts. lib/checkout/split.ts — idempotent vendor split for all 5 vendor types. app/api/checkout/route.ts. app/api/webhooks/stripe/route.ts. (one pays all). app/api/profile/prompt/route.ts + lib/intent/profileQuery.ts.	app/api/stage/sse/route.ts — persistent SSE per participant. app/api/stage/lock/ + stream/ routes. lib/sse/ broadcast layer + reconnection. lib/gifts/ — GiftOrder model, token, create + redeem. app/api/gifts/ + app/gift/[token]/. lib/privacy/ — public/private profiles, data sharing rules, follow request system. app/api/follow/.	Checkout UI — SingleCheckout, MultiCheckout, mode selector. Confirmation page — all vendor refs in one card. Gift Link sender checkout flow + gift reveal page. Follow request UI (pending, accept, decline). Privacy settings panel. Invite link landing page (pre-built Stage for non-iAM @mention).
W4 Hardening + Demo	Intent parser edge cases — ambiguous dates, missing destinations, multi-city. Flight offer expiry handling + re-fetch flow. Ranking unit tests — prove bid cannot bypass gate. app/[handle]/page.tsx polish. Profile prompt quality review. Demo environment — clean seed, reliable E2E flow.	All 5 vendor order creation paths hardened. Gift link edge cases — expiry (3 days), failed payment, already-redeemed token. SSE reconnection + Redis state rehydration under load. Follow request notifications. Profile privacy enforcement — parse only allowed data. End-to-end test suite across all 7 services.	Mobile responsive pass — all screens. Error + empty states per service row. Order history UI. Beta user onboarding screens. Investor-quality demo screens. Component documentation for Maaz + Nekha handoff.

4. Detailed Weekly Breakdown

4.1 Maaz

Week 1 — Type contracts + intent foundation

- lib/intent/types.ts — define and merge ParsedIntent, ActivityType, Participant
- lib/checkout/types.ts — define and merge CartItem, StageCart, VendorType, PendingOrder
- lib/intent/parser.ts — extend to handle all 7 activity type signals (flights, stays, cars, experiences, restaurants, weather, maps)
- lib/services/flights/ — complete partial integration: offer expiry guard, error wrapping, caching hook
- lib/ranking/gate.ts — relevance threshold check, returns boolean, no bid logic here
- lib/ranking/scorer.ts — stub with clear interface (Nekha's assembler will call this)
- lib/payments/stripe.ts — Stripe client, createPaymentIntent(), base webhook signature verify

Week 2 — Participant resolution + profile foundation

- lib/intent/participants.ts — resolve @handles from MongoDB, fetch intent graphs, generate invite Stage links for non-iAM users
- app/api/intent/route.ts — validates ParsedIntent, calls assembler (Nekha's module), returns staged
- lib/profile/docParser.ts — parse uploaded DOCX/PDF, map fields to IntentGraph schema
- app/api/profile/upload/route.ts — Blob upload, trigger docParser, upsert stageProfiles collection
- lib/ranking/scorer.ts — full implementation: Intent Fit, User Fit, Outcome History scoring logic
- lib/ranking/ranker.ts — bid applied within already-qualified set only, unit tests

Week 3 — Checkout + payment modes + profile prompt

- lib/checkout/pendingOrder.ts — create, persist, expiry management
- lib/checkout/split.ts — executeVendorSplit() covering all 5 VendorTypes, idempotency guard via processedSplits collection
- app/api/checkout/route.ts — validate cart, create PendingOrder, return Stripe client_secret
- app/api/webhooks/stripe/route.ts — payment_intent.succeeded handler, calls split.ts and per-vendor order modules
- Stripe SplitPayment modes — one_pays_all, split_equally (Stripe Payment Links per participant), pay_your_own (parallel intents)
- app/api/profile/prompt/route.ts — receive visitor query, call profileQuery.ts, return text response

- lib/intent/profileQuery.ts — query answering scoped to profile owner's intent graph + uploaded document context

Week 4 — Hardening + demo

- Intent parser edge cases — missing destination fallback, ambiguous date parsing, multi-city trips
- offer expiry — detect expired offer at checkout initiation, surface alternative from same row
- lib/ranking/ unit tests — prove a bid=MAX card with Intent Fit=0 is excluded. Prove a bid=0 card with high Intent Fit ranks above a bid=MAX card with low Intent Fit
- app/[handle]/page.tsx — profile page server component, pulls stageProfiles, enforces Nekha's privacy middleware
- lib/intent/graph.ts — post-booking outcome signals: destination, spend tier, activity type, group size
- Demo seed script — clean Paris trip scenario, pre-resolved @handles, reliable flow from prompt to confirmation

4.2 Nekha

Week 1 — DB schema + Duffel complete + cache layer

- lib/db/mongo.ts — all collections: users, stageProfiles, intentGraphs, stages, stageCarts, pendingOrders, processedSplits, orders, giftOrders, followRequests. ensureIndexes() for all. Export getDb() and COLLECTIONS constants
- lib/services/duffel/client.ts — shared HTTP client with auth headers, retry logic, timeout config
- lib/services/stays/ — hotel search (destination, dates, guest count), room offer retrieval, room config for party size
- lib/services/cars/ — car search (pickup location, dates, category filter), offer retrieval, driver details structure
- lib/cache/ — Redis wrapper: get/set/invalidate with TTL per service type, keyed by stageId + serviceType

Week 2 — Remaining 3 services + stage assembly

- lib/services/viator/ — product search by destination + date range + category, availability check, booking payload structure
- lib/services/opentable/ — restaurant search by location + date + party size + cuisine, availability, reservation payload. Party size = participants.length + 1

- lib/services/maps/ — Google Maps Places API: nearby search for malls, POIs, shopping areas. Static results — cache aggressively (6h TTL)
- lib/stage/assembler.ts — takes MergedStageContext, fires Promise.all() across all 7 services, streams rows as each resolves, calls gate.ts per card
- lib/stage/merge.ts — average User Fit across participant intent graphs, penalise cards scoring below threshold for any single participant, enforce party size on hotel + restaurant cards

Week 3 — SSE + collaboration + gifts + privacy

- lib/sse/ — SSE connection manager: per-participant event streams, broadcast lock events, stage update pushes, gift notifications
- app/api/stage/sse/route.ts — persistent connection endpoint, Redis pub/sub for cross-instance delivery
- app/api/stage/lock/route.ts — lock a card: create CartItem, update stageCart in MongoDB, broadcast via SSE
- app/api/stage/stream/route.ts — stream assembled Stage rows progressively, one chunk per service as it resolves
- lib/gifts/ — GiftOrder model, token generation (nanoid), createGiftOrder(), redeemGiftOrder(address), expiry at 3 days
- app/api/gifts/create/route.ts — validate item, save Stripe paymentMethodId, generate token, dispatch notification
- app/api/gifts/redeem/route.ts — validate token + expiry, store address, trigger Stripe charge, create vendor order
- app/gift/[token]/page.tsx — server component: fetch gift order, render reveal page with item + message + address form
- lib/privacy/ — ProfilePrivacy model: public | private | followers_only. Data sharing rules: parse only fields user set as shareable in their profile settings
- app/api/follow/ — followRequest create (POST), accept/decline (PATCH), list incoming/outgoing (GET). Notify sender via SSE on accept

Week 4 — Edge cases + reliability + invite links

- Invite link generation — when @mentioned user is not on iAM: generate signed URL, pre-build Stage around initiator's intent, store in Redis with 48h TTL. Link lands on a live Stage (not a sign-up wall). On sign-up, layer in new user's preferences and push Stage update to existing participants via SSE
- All 5 vendor order creation paths — duffel order.ts (flight + hotel + car separately), viator order.ts, opentable order.ts — each called from Maaz's webhook handler

- Gift link expiry — background job (Vercel cron): scan giftOrders where status=pending_address AND createdAt < now-3days, set status=expired, release any Stripe payment method holds
- SSE reconnection — client-side exponential backoff, reconnect on drop, rehydrate stage state from Redis on reconnect, replay any missed lock events since last event ID
- Profile privacy enforcement — middleware on app/[handle]/page.tsx: check profile.privacy, check follow status, return only allowed fields based on user's data sharing config
- End-to-end tests — full Paris trip flow across all 7 services in sandbox, gift link create-to-redeem, follow request accept-to-SSE-notify

5. Ashish — Design Task Table

Ashish and Samantha own 100% of the UI. Engineers wire up data but do not touch styling, layout, or component structure. All components must support dark mode via next-themes — this is non-negotiable per the tech stack.

Priority	Component / Screen	Details	Wk
P1	Design system tokens	Font scale, colour palette (light + dark), spacing scale, border radius, elevation. Dark mode CSS variables for every token. Component baseline: Button, Input, Badge, Avatar, Card shell.	1
P1	Stage row anatomy	Row header (service label, icon, count), Card grid layout, Card variants per service type (flight card, hotel card, car card, experience card, restaurant card, weather card, maps card). Each card has one CTA button.	1
P1	Card — Flights	Airline logo, duration, price. Book CTA. Locked state (blue overlay, lock icon, locker's avatar).	1
P1	Card — Stays	Hotel image, name, star rating, location, price per night, total. Reserve CTA. Locked state.	1
P1	Card — Cars	Car image, category, provider logo, daily rate, total. Book CTA. Locked state.	1

P1	Onboarding questions	3-4 questions: accommodation preference, food style, activity preference. Minimal, conversational layout. Progress indicator. Skippable with clear skip CTA. Response stored as Intent Graph seed.	1
P1	Card — Experiences (Viator)	Activity image, title, duration, rating, group size compatibility, price. Book CTA. Locked state.	2
P1	Card — Restaurants (OpenTable)	Restaurant image, name, cuisine, rating, available time slots, party size. Reserve CTA. Locked state.	2
P1	Row — Weather	Compact row: destination, date range, daily forecast icons, temperature range, packing suggestion chip. No CTA — informational.	2
P1	Row — Maps & Shopping	Compact card list: mall/area name, distance, opening hours, Google Maps deep link. No book CTA — discovery only.	2
P1	Stage — progressive loading	Per-row skeleton state (not full-page spinner). Each row resolves independently. Row slides in when its service responds. Never a blank white screen.	2
P1	Participant presence bar	Horizontal avatar row — shows who is in the Stage. Active, idle, and offline states. Lock attribution — small avatar on locked card showing who locked it.	2
	Detail Window - Each Row Item		
P1	Profile page	Full-width header image, circular avatar, display name, @handle, short bio, location, follow button. Prompt input pinned at bottom. Scroll content area for future cards. Public and private state variants.	2
P1	Document upload UX	Drag-and-drop zone, file type + size validation, upload progress bar, parse success / error state, link to sample document download.	2

P1	Checkout — SingleCheckout	One item summary, price breakdown (item + iAM fee), Stripe card input, Pay button. Success -> confirmation card.	3
P1	Checkout — MultiCheckout	Grouped cart (items by service type), per-vendor subtotals, combined total with iAM fee, single Pay button.	3
P1	Confirmation page	Combined card: all booking references, vendor logos, dates, party details. Share/download itinerary action. SSE-pushed to all participants simultaneously.	3
P1	Gift Link — sender flow	Buy as Gift modal: item summary, date/spec fields, personal message input, card entry. Address field shown as locked with recipient avatar placeholder.	3
P1	Gift Link — gift reveal page	Animated gift reveal: sender's avatar, gift item details, personal message, address input form. Minimal, celebratory but clean.	3
P1	Gift Link — confirmations	Sender view: gift sent, awaiting address. Post-redeem: gift confirmed, delivery details. Recipient view: gift confirmed, booking reference.	3
P1	Follow request UI	Follow button states: Follow / Requested / Following / Unfollow. Incoming requests list: accept / decline. Follower / following count on profile. Request notification toast.	3
P1	Privacy settings panel	Public / Private / Followers only toggle. Data sharing granularity: which intent graph fields are shareable (destinations, preferences, booking history). Clear labelling.	3
P1	Invite link landing page	Pre-built Stage shown immediately — no sign-up wall. Sign-up prompt floats at top as a non-blocking banner. Stage is usable before signing up (guest view).	3
P2	Order history	Bookings, gifts sent, gifts received as separate tabs. Booking card: trip summary, vendor logos, booking refs, date, amount. Status badge.	4

P2	Profile prompt response	Text response below prompt input. Thread layout for follow-up questions. Loading state (streaming). Post-MVP placeholder for card responses.	4
P2	Error states — per service row	Service-specific error card: Flights unavailable — try a different date. Retry button. Never crashes the whole Stage.	4
P2	Empty states	No results for a row, no bookings in history, no followers yet, no gifts sent.	4
P2	Mobile responsive — full pass	All screens: Stage, profile, gift reveal, checkout, confirmation, order history. Minimum 375px viewport.	4
P2	Investor demo screens	High-fidelity static screens: Paris Stage with 3 locked items, gift reveal animation frame, split payment selector. For use in pitch deck.	4

6. Merge Conflict Prevention — Working Rules

The 3 rules that eliminate 90% of conflicts:

1. Freeze shared types on Day 2 — both engineers pull before writing any feature code. 2. One engineer per directory — if a directory has an owner, the other engineer never creates files there. 3. Cross-boundary changes go through a PR comment first — even a one-line Slack message. No surprise commits in another engineer's files.

Scenario	Wrong approach	Right approach
Nekha needs a new CartItem field	Edits lib/checkout/types.ts directly	Opens a PR discussion with Maaz. Maaz adds the field and merges. Nekha pulls.
Maaz needs a new MongoDB collection	Adds schema to lib/db/mongo.ts directly	Opens a PR discussion with Nekha. Nekha adds collection + index. Maaz pulls and uses <code>getDb()</code> .
Ashish needs a store value for the UI	Edits store/stageStore.ts directly	Describes the value needed to Nekha. Nekha adds a getter. Ashish pulls and imports.

Both need to update lib/mail.ts	Both edit at the same time	Agree who owns it for this sprint (Maaz). Nekha passes data to Maaz's mail functions.
Nekha adds a new Viator service response shape	Changes shared lib/intent/types.ts ActivityType	Adds new type inside lib/services/viator/types.ts only. Does not touch shared contracts.

Branch strategy	PR rule	Review
main — production only	Never commit to main directly	Both engineers review shared contract PRs before merge
maaz/feature-name — Maaz's branches	Maaz merges his own PRs after self-review unless touching SHARED files	Nekha reviews any PR that touches a SHARED file
nekha/feature-name — Nekha's branches	Nekha merges her own PRs after self-review unless touching SHARED files	Maaz reviews any PR that touches a SHARED file
Daily sync — 30 min standup	What did you merge? What will you merge today?	Flag any cross-boundary need early — same day, not end of week

iAM

Intent becomes action.

4 weeks. 7 services. One file per owner.

The north star:

Payment can shift position. It cannot create relevance. If a feature makes you unsure — come back to this line.